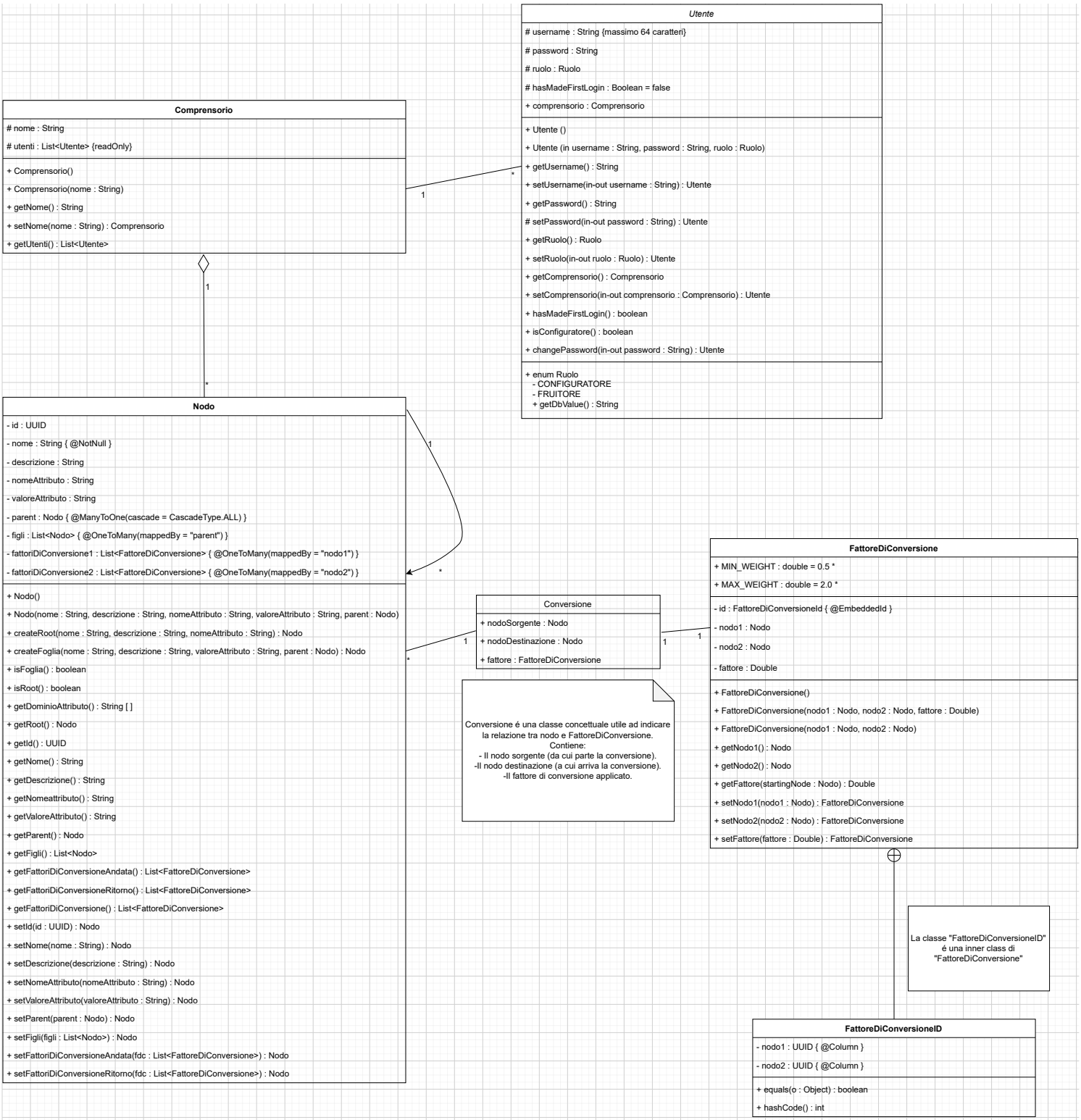

Utente
username : String {massimo 64 caratteri}
password : String
ruolo : Ruolo
hasMadeFirstLogin : Boolean = false
+ comprensorio : Comprensorio
+ Utente ()
+ Utente (in username : String, password : String, ruolo : Ruolo)
+ getUsername() : String
+ setUsername(in-out username : String) : Utente
+ getPassword() : String
setPassword(in-out password : String) : Utente
+ getRuolo() : Ruolo
+ setRuolo(in-out ruolo : Ruolo) : Utente
+ getComprensorio() : Comprensorio
+ setComprensorio(in-out comprensorio : Comprensorio) : Utente
+ hasMadeFirstLogin() : boolean
+ isConfiguratore() : boolean
+ changePassword(in-out password : String) : Utente
+ enum Ruolo
- CONFIGURATORE
- FRUITORE
+ getDbValue() : String
Comprensorio
nome : String
utenti : List<Utente> {readOnly}
+ Comprensorio()
+ Comprensorio(nome : String)
+ getNome() : String
+ setNome(nome : String) : Comprensorio
+ getUtenti() : List<Utente>
1
*
1
*
Nodo
- id : UUID
- nome : String { @NotNull }
- descrizione : String
- nomeAttributo : String
- valoreAttributo : String
- parent : Nodo { @ManyToOne(cascade = CascadeType.ALL) }
- figli : List<Nodo> { @OneToMany(mappedBy = "parent") }
- fattoriDiConversione1 : List<FattoreDiConversione> { @OneToMany(mappedBy = "nodo1") }
- fattoriDiConversione2 : List<FattoreDiConversione> { @OneToMany(mappedBy = "nodo2") }
+ Nodo()
+ Nodo(nome : String, descrizione : String, nomeAttributo : String, valoreAttributo : String, parent : Nodo)
+ createRoot(nome : String, descrizione : String, nomeAttributo : String) : Nodo
+ createFoglia(nome : String, descrizione : String, valoreAttributo : String, parent : Nodo) : Nodo
+ isFoglia() : boolean
+ isRoot() : boolean
+ getDominioAttributo() : String [ ]
+ getRoot() : Nodo
+ getId() : UUID
+ getNome() : String
+ getDescrizione() : String
+ getNomeattributo() : String
+ getValoreAttributo() : String
+ getParent() : Nodo
+ getFigli() : List<Nodo>
+ getFattoriDiConversioneAndata() : List<FattoreDiConversione>
+ getFattoriDiConversioneRitorno() : List<FattoreDiConversione>
+ getFattoriDiConversione() : List<FattoreDiConversione>
+ setId(id : UUID) : Nodo
+ setNome(nome : String) : Nodo
+ setDescrizione(descrizione : String) : Nodo
+ setNomeAttributo(nomeAttributo : String) : Nodo
+ setValoreAttributo(valoreAttributo : String) : Nodo
+ setParent(parent : Nodo) : Nodo
+ setFigli(figli : List<Nodo>) : Nodo
+ setFattoriDiConversioneAndata(fdc : List<FattoreDiConversione>) : Nodo
+ setFattoriDiConversioneRitorno(fdc : List<FattoreDiConversione>) : Nodo
1
*
Conversione
+ nodoSorgente : Nodo
+ nodoDestinazione : Nodo
+ fattore : FattoreDiConversione
1
1
1
1
Conversione é una classe concettuale utile ad indicare la relazione tra nodo e FattoreDiConversione.
Contiene:
- Il nodo sorgente (da cui parte la conversione).
-Il nodo destinazione (a cui arriva la conversione).
-Il fattore di conversione applicato.
FattoreDiConversione
+ MIN_WEIGHT : double = 0.5 *
+ MAX_WEIGHT : double = 2.0 *
- id : FattoreDiConversioneId { @EmbeddedId }
- nodo1 : Nodo
- nodo2 : Nodo
- fattore : Double
+ FattoreDiConversione()
+ FattoreDiConversione(nodo1 : Nodo, nodo2 : Nodo, fattore : Double)
+ FattoreDiConversione(nodo1 : Nodo, nodo2 : Nodo)
+ getNodo1() : Nodo
+ getNodo2() : Nodo
+ getFattore(startingNode : Nodo) : Double
+ setNodo1(nodo1 : Nodo) : FattoreDiConversione
+ setNodo2(nodo2 : Nodo) : FattoreDiConversione
+ setFattore(fattore : Double) : FattoreDiConversione
La classe "FattoreDiConversioneID" é una inner class di "FattoreDiConversione"
FattoreDiConversioneID
- nodo1 : UUID { @Column }
- nodo2 : UUID { @Column }
+ equals(o : Object) : boolean
+ hashCode() : int